

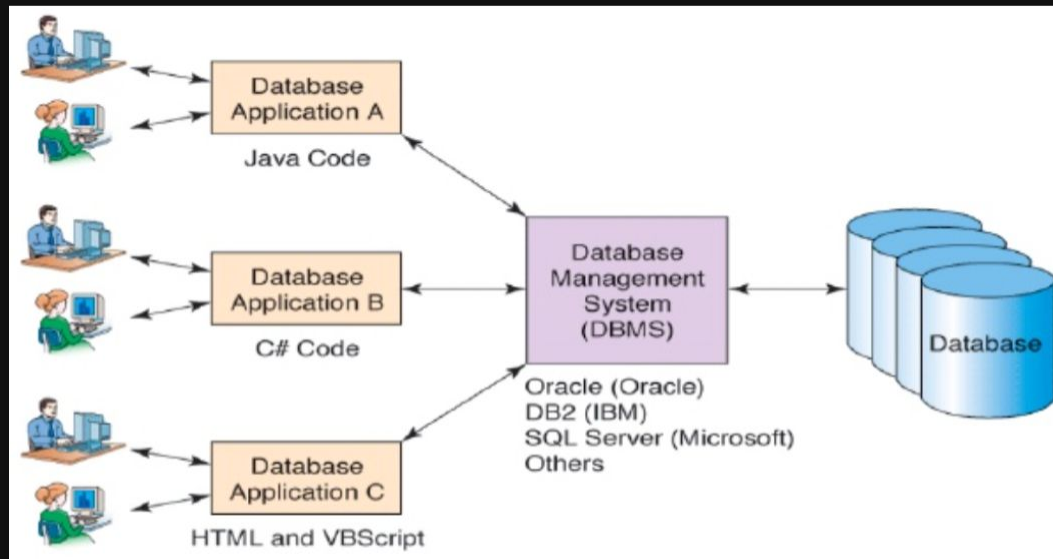
Relational Databases

Created by <Academia de Código_>

Relational Database Management Systems (RDBMS)

What is a RDBMS?

It's a software package designed to manipulate data in a relational database



But what is a database?

It's nothing more than a set (usually big) of related information, which describes activities of the real world.

Compared to paper, a database system is able to retrieve data more quickly, index it in multiple ways and deliver up-to-date information to its users

Modern database systems can manage terabytes of data spread across many fast-access disk drives, holding tens of gigabytes of data in high-speed memory

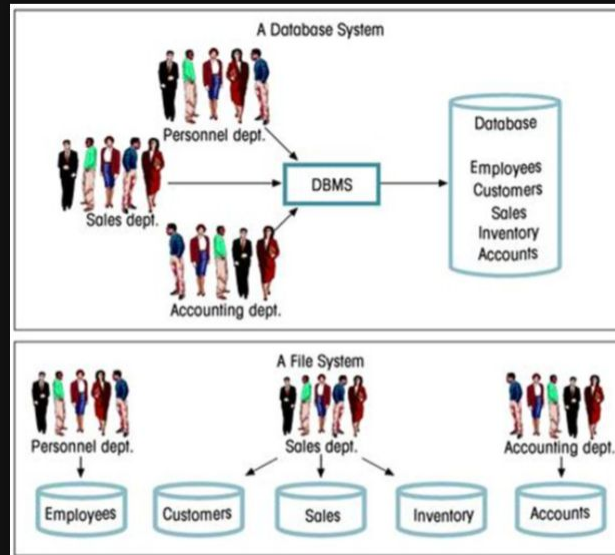
Examples of Databases:

- Telephone books - contains information concerning all the subscribers
- Student database - contains information concerning all the students in a school
- Library database - contains information concerning all the books in a library

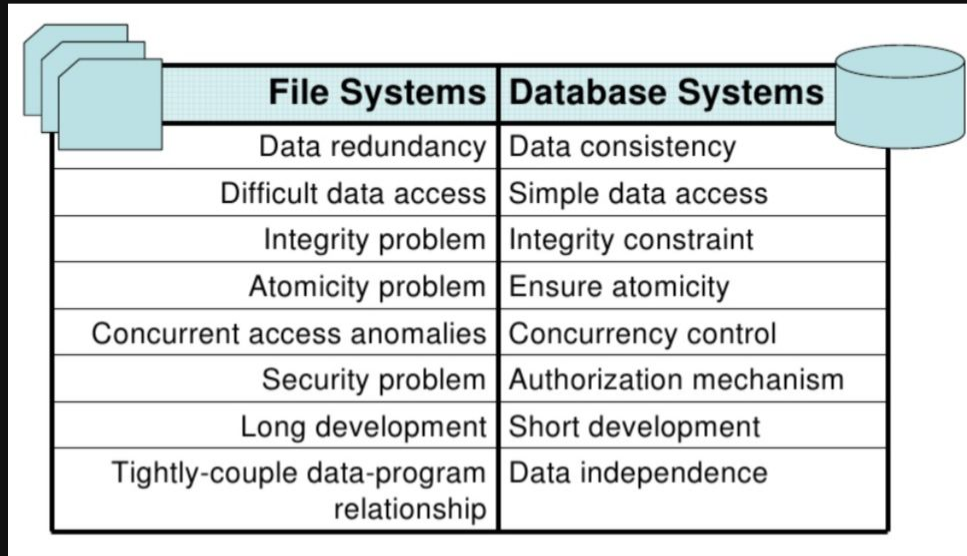
Is there another way to persist data?

Yes, there are other ways...

For example, we could use plain files



Advantages of using a DBMS



File Systems	Database Systems
Data redundancy	Data consistency
Difficult data access	Simple data access
Integrity problem	Integrity constraint
Atomicity problem	Ensure atomicity
Concurrent access anomalies	Concurrency control
Security problem	Authorization mechanism
Long development	Short development
Tightly-couple data-program relationship	Data independence

Disadvantages of using a DBMS

Do not use a DBMS when:

- The DBMS does not support the types of queries or functionality required by the application
- The advanced DBMS functionality is not needed

DBMS introduces overhead:

- Installation and configuration overhead
- Operating overhead - concurrency control, atomicity, etc.
- Cost - hardware, software, maintenance and support

What are relational databases?

A database that implements stored data in a series of two dimensional relations (tables) that are related to one another

Student Table

 ID No.	Name	D.o.B.	Phone	Class
356	Jess	3 Mar 1995	7564356	5B
412	Hamad	12 Nov 1994	7465846	5B
459	Sita	9 Jan 1994	8565634	6Y
502	Hamad	3 Mar 1995	6554546	5B

Class Table

 Class	Tutor	Room
5B	Mr Noggin	56
6Y	Ms Take	18

The Class field acts as a **relationship** (link) between the tables

Relational vs Non-relational

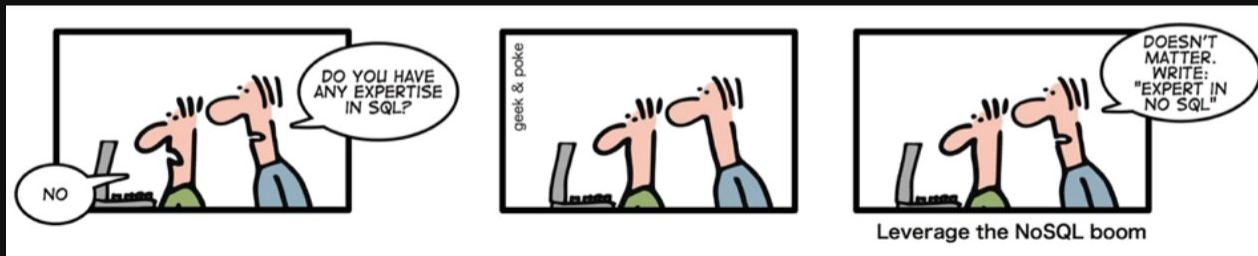
Relational

- Data is relational in nature - customer purchases products, products have a supplier and a manufacturer
- Need to locate relevant information quickly
- Scalability, reliability, ACID (Atomicity, Consistency, Isolation, Durability) compliance
- Reporting or intelligence tools to work out business problems
- Results based on multiple search condition

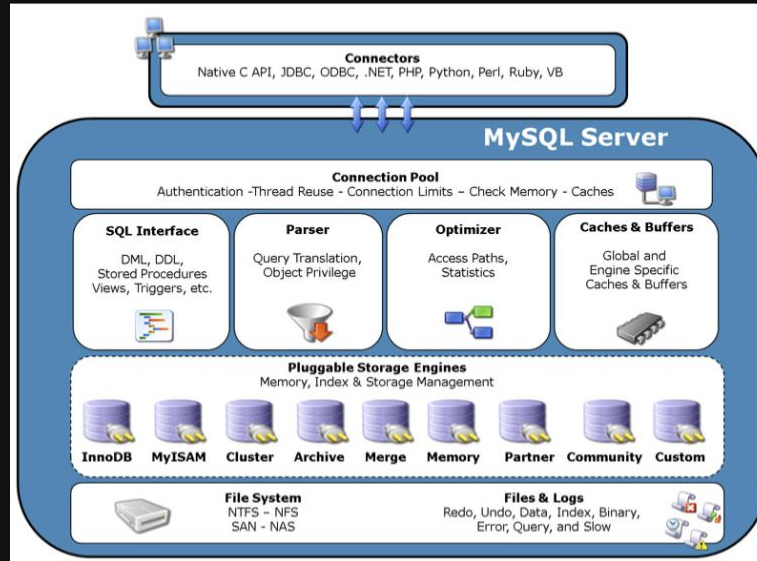
Non-relational

- Lots of unstructured data that needs to be stored
- Scalability and speed

In a nutshell: Flexibility vs Performance



Most Popular Open Source Database



- Has connectors for a number of programming languages
- Works on just about all operating systems

Hello Database

```
// Database connection
$ mysql -u <username> -h <host> -p [database]

/* SQL STATEMENTS BELOW */

// Database creation
> CREATE DATABASE boat_booking;

// Start using Database
> USE boat_booking;

// Delete database #PROCEEDWITHCAUTION
> DROP DATABASE [if exists] boat_booking;

// List databases
> SHOW DATABASES;
```

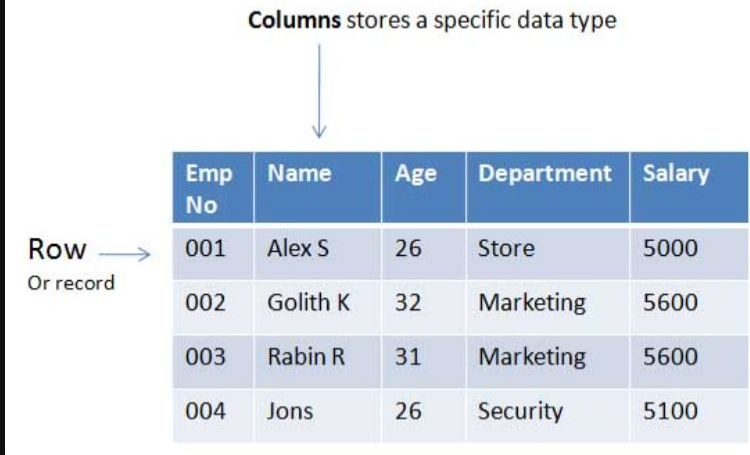
Tables



How information is organized in a relational database

Database Relations (Tables)

Columns stores a specific data type



Emp No	Name	Age	Department	Salary
001	Alex S	26	Store	5000
002	Golith K	32	Marketing	5600
003	Rabin R	31	Marketing	5600
004	Jons	26	Security	5100

- Name - each table should have a unique name
- Fields or Attributes - each column in a table
- Records or Tuples - each row in a table

Attribute Data type

MySQL Type	Java Type
-----	-----
CHAR	String
VARCHAR	String
LONGVARCHAR	String
NUMERIC	java.math.BigDecimal
DECIMAL	java.math.BigDecimal
BIT	boolean
TINYINT	byte
SMALLINT	short
INTEGER	int
BIGINT	long
REAL	float
FLOAT	double
DOUBLE	double
BINARY	byte[]
VARBINARY	byte[]
LONGVARBINARY	byte[]
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp

Creating a table

```
/* SQL STATEMENTS BELOW */  
// Generic table creation  
CREATE TABLE generic_table(  
    column1 DATATYPE [ CONSTRAINT ] ,  
    column2 DATATYPE [ CONSTRAINT ]  
);
```

Constraints

Constraint	Meaning
NOT NULL	Column will not hold null values
UNIQUE	Column will not hold already existing values
AUTO_INCREMENT	Column will automatically increment
DEFAULT <value>	Set the column's default value

```
CREATE TABLE example_table(  
  id INTEGER,  
  name CHAR(15) NOT NULL UNIQUE,  
  PRIMARY KEY (id)  
);
```

```
CREATE TABLE example_table(  
  id INTEGER AUTO_INCREMENT,  
  PRIMARY KEY (id)  
);
```

The fishing village



We need to commission a new software to help us register sailors and boats and manage our day to day operations. All sailors are registered with a unique number identifier and have a name and age. We have a ledger for the boats. They have a name, a color and also a unique number identifier.

{ Hands On }

Creating the sailors table

```
CREATE TABLE sailors(  
  id INTEGER NOT NULL UNIQUE,  
  name CHAR(15) NOT NULL,  
  age INTEGER NOT NULL  
);
```

{ Exercise }

Create the boats table

Database Keys

Refers to specific table attributes

Used to sort and/or uniquely identify table records

Used to establish and identify relationships between tables

The most important key types are:

- Primary Key
- Foreign Key



Primary Key

Every table must have a primary key, a set of attributes that will uniquely identify a record.

Brand	Owner	BI	License Plate	Year	Model
Audi	Pedro	12345	AS-67-DF	2004	TT
Honda	Ferrão	54321	AS-68-LS	2006	Civic Coupe
Jeep	Campelo	22222	GD-64-69	1998	
Fiat	Mário	66666	56-AF-72	2013	Punto
Audi	Pedro	12345	KF-13-HF	2004	R8 Spyder
McLaren	Campelo	22222	85-AD-16		650s spider
Fiat	Mário	66666	67-AS-DF	2006	Panda
Ford	Ferrão	98765	16-HY-16	2010	Fiesta
Jaguar	Ferrão	54321	GH-73-MP	2000	X-Type

In this case what would be a good primary key?

Primary Key - Composite

In case there is no single attribute for a primary key, we can use a composite - several attributes

Name	Age	Favorite Food	Birthdate	Favorite Movie
João	20	Frango	05-08-96	Star Trek
Pedro	30	Frango	10-10-86	Star Trek
Rui	30	Polvo	10-10-86	Batman vs Superman
Pedro	30	Frango	9-02-86	Star Trek
Mário	30	Gato	24-02-86	The Matrix
Mário	40	Peixe	9-10-76	Jason Bourne
Campelo	41	Polvo	04-06-75	The Matrix
Rui Ferrão	41	Lulas	04-06-75	Ben-Hur

In this case what would be the primary key?

Primary Key

We can add the primary key as a constraint on the attribute or on the table

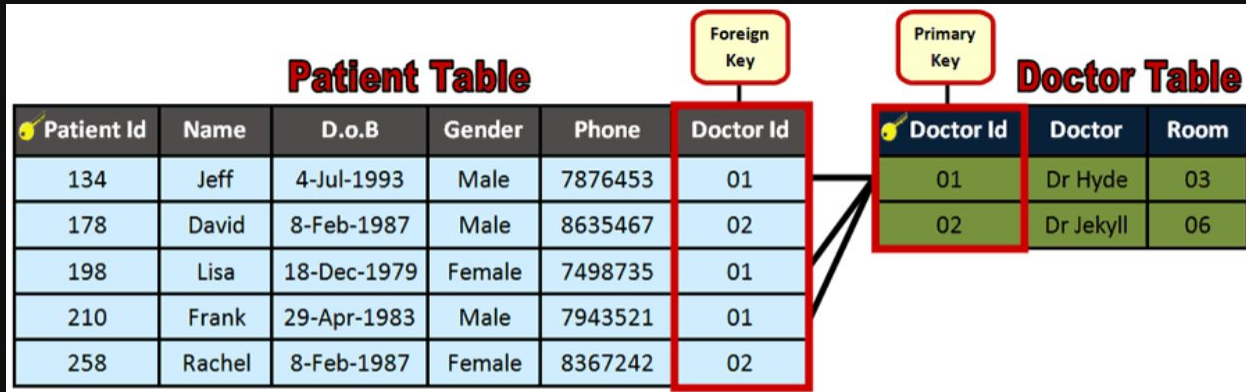
```
CREATE TABLE mytable(  
    column1 DATATYPE [ NOT NULL | UNIQUE | PRIMARY KEY],  
    column2 DATATYPE [ NOT NULL | UNIQUE | PRIMARY KEY]  
);
```

```
CREATE TABLE mytable(  
    column1 DATATYPE [ NOT NULL | UNIQUE ],  
    column2 DATATYPE [ NOT NULL | UNIQUE ],  
    PRIMARY KEY (column1,column2)  
);
```

Foreign keys

A foreign key is an attribute on a table that points to a primary key in another table.

Useful to enforce relationships and to relate information.



Foreign keys setup

```
CREATE TABLE mytable(  
    column1 DATATYPE [ NOT NULL | UNIQUE | PRIMARY KEY],  
    column2 DATATYPE [ NOT NULL | UNIQUE | DEFAULT <value>  
    | FOREIGN KEY (column2) REFERENCES mytable2(column1) ]  
);
```

```
CREATE TABLE mytable(  
    column1 DATATYPE [ NOT NULL | UNIQUE],  
    column2 DATATYPE [ NOT NULL | UNIQUE | DEFAULT <value>],  
  
    PRIMARY KEY (column1),  
    FOREIGN KEY (column2) REFERENCES mytable2(column1)  
);
```

Table Relationships



A relationship exists between two tables when you associate the records of the first table with those of the second.

Crucial to data integrity because they reduce redundant data and eliminate duplicate data

Cardinality

The type of a relationship between two tables is known as cardinality.

Three specific types of relationship can exist between tables:

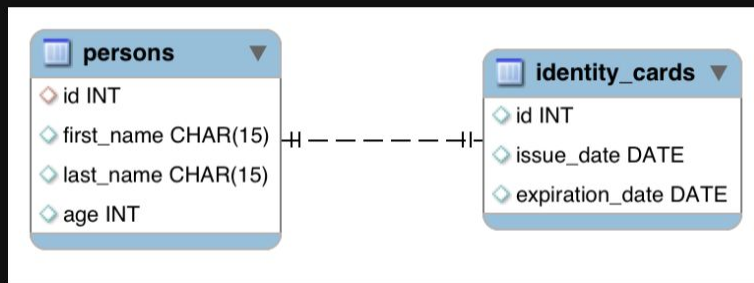
1. one-to-one
2. one-to-many
3. many-to-many

zero or more	
one or more	
one and only one	
zero or 1	

One-to-One

A pair of tables has a one-to-one relationship when a single record in the first table is related to only one record in the second table, and a single record in the second table is related to only one record in the first table

In this type of relationship one table serves as a parent table and the other serves as a child table

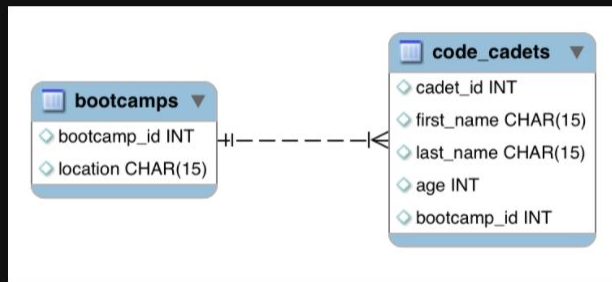


This is a special type of relationship because it is the only one in which both tables may actually share the same primary key

One-to-Many

A pair of tables has a one-to-many relationship when a single record in the first table can be related to many records in the second table, but a single record in the second table can be related to only one record in the first table

The table on the one side of the relationship is the parent table and the table on the many side is the child table

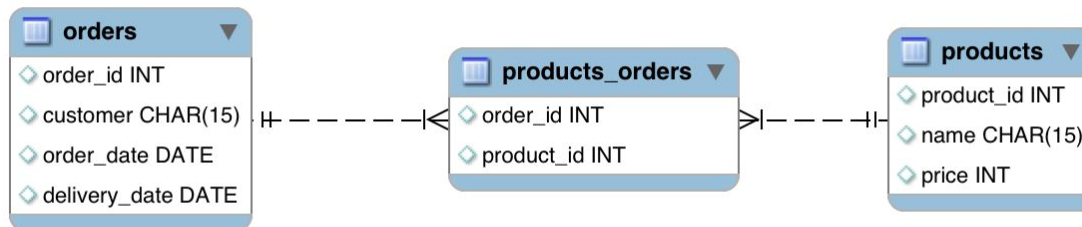


This is the most common relationship that exists between a pair of tables in a database

Many-to-Many

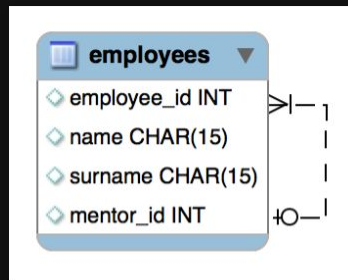
A pair of tables has a many-to-many relationship when a single record in the first table can be related to many records in the second table and a single record in the second table can be related to many records in the first table

This relationship is established using a linking table



Self-Referencing Relationship

Does not exist between a pair of tables, but between the records within a table



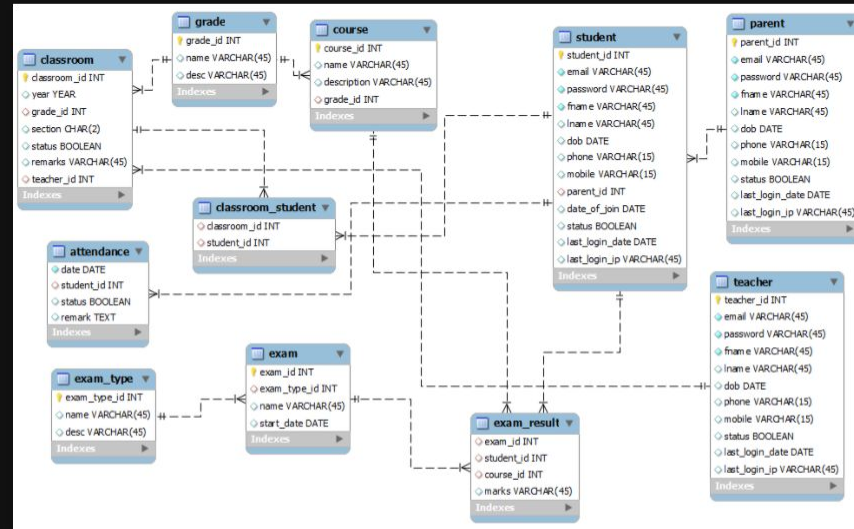
{ Exercise }

Update the database to support a reservation system
where sailors can book a boat for the whole day

Schema

Logical definition of a table, consisting of the name and type of each attribute

A database schema is the collection of table schemas for the whole database



Database schema for a school management software

Changing table schema

Remove columns

```
ALTER TABLE mytable  
DROP mycolumn
```

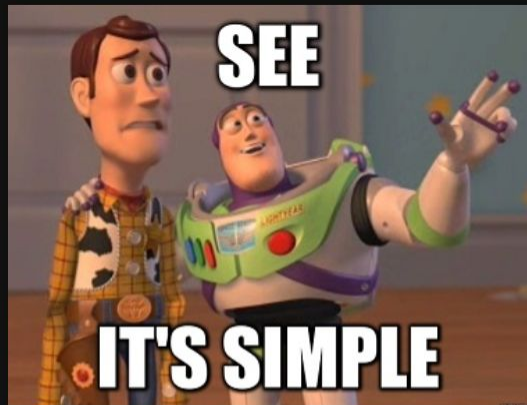
Insert columns

```
ALTER TABLE mytable  
ADD mycolumn DATATYPE [ CONSTRAINTS ]
```

All new records will have a null value if the attribute is not specified

Removing tables

```
DROP TABLE mytable
```



Structured Query Language (SQL)

Programming language designed for managing data held in a RDBMS

It is declarative (not procedural), which means we just tell what we want without having to supply a step-by-step procedure on how to do it



Table Operations

- Select
- Insert
- Delete
- Update

In computer programming, these four basic functions of persistent storage are called CRUD



How to insert data in a table?

```
INSERT INTO mytable(column1, column2)
VALUES (column1_value, column2_value);
```

Example:

```
INSERT INTO sailors(id, name, rating, age)
VALUES (15, 'Andreia', 55.44, 30);
```

How to query for data in a table?

Basic syntax:

```
SELECT column1, column2 FROM mytable;
```

Example selecting all columns:

```
SELECT * FROM sailors;
```

How to delete data from a table?

```
DELETE FROM mytable WHERE column=column_value;
```

Example:

```
DELETE FROM sailors WHERE sailor_id=15;
```

How to update data in a table?

```
UPDATE table  
SET column1=column1_value  
WHERE column2=column2_value;
```

Example:

```
UPDATE sailors  
SET age=25  
WHERE sailor_id=1;
```

Integrity Restrictions

An integrity restriction is a specified condition over the schema in order to restrict the data that can be stored in a table

- Domain integrity
- Entity integrity
- Referential integrity

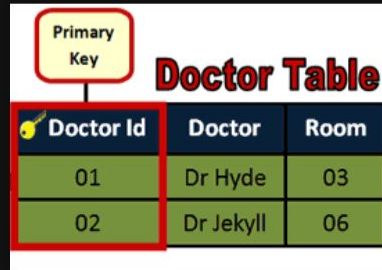
Domain Integrity

A domain defines the possible values for an attribute

- Type - basic data types for numbers, strings, date and time
- Length - number of digits or characters in a value
- Range - lower and upper boundaries of the values
- Constraints - special restrictions on allowable values
- Null support - whether the attribute can have a null value
- Default value - The value an attribute will have if a value is not supplied

Entity Integrity

Concerns the concept of a Primary Key.



The diagram shows a table titled "Doctor Table". A yellow box labeled "Primary Key" with a key icon points to the "Doctor Id" column. The "Doctor Id" column is highlighted with a red border. The table has three columns: "Doctor Id", "Doctor", and "Room". The data rows are:

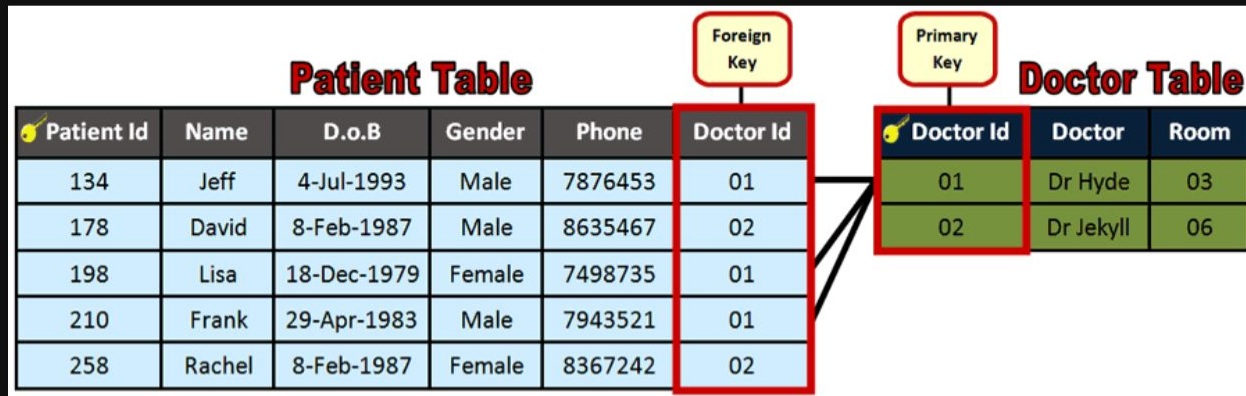
Doctor Id	Doctor	Room
01	Dr Hyde	03
02	Dr Jekyll	06



- Every table must have a primary key
- The column or columns chosen to be the primary key should be unique and not null.
- When more than one column makes up the primary key, it is referred to as a concatenated primary key
- Usually an integer attribute with the name id

Referential Integrity

Concerns the concept of a Foreign Key.



- A foreign key must reference another column (usually the primary key from another table)
- If the data in one table is changed, related tables will be verified to ensure consistency

Referential Integrity

on delete *action*

on update *action*

action is one of :

- cascade - deletes/updates the foreign key entries that point to the referred record;
- set null - makes null the foreign key entries that point to the referred record;
- set default - sets to the default value the foreign key entries that point to the referred record;
- no action - prevents the entry that is being referred to from being deleted/updated. This is the default behaviour...

Referential Actions

```
CREATE TABLE reserves(  
    sailor_id INTEGER,  
    boat_id INTEGER,  
    day DATE,  
    PRIMARY KEY(boat_id, day),  
    FOREIGN KEY(sailor_id) REFERENCES sailors(id) ON DELETE CASCADE,  
    FOREIGN KEY(boat_id) REFERENCES boats(id) ON DELETE CASCADE  
);
```

Database Queries

How to access the data?

"What's the fraction of students in the DB's discipline that had a grade superior to 16?"

It's this type of questions that we will learn how to answer.

Select Queries

A select always returns a result-set and is composed by two major parts

- Projection - Which columns to return in the result set
- Selection - Which rows to return in the result set

Select the name of all sailors with age below 30

Projection: name

Selection: age < 30

```
SELECT sailor_name FROM sailors WHERE age < 30
```

{ Hands On }

Populate the database with data

{ Exercise }

Perform selection queries

What are the names of all boats?

List all red boats

List all sailors with age between 20 and 30

List the id and name of sailors with less than 50 years and id higher than 5

Using more tables

a join

A join is used to combine rows from two or more tables, based on a common field

- INNER JOIN - all rows when there is at least one match in BOTH tables
- RIGHT JOIN - all rows from the right table, and the matched rows from the left table
- LEFT JOIN - all rows from the left table, and the matched rows from the right table
- FULL JOIN - all rows when there is a match in ONE of the tables

We will just focus on the INNER JOIN

Using two tables

What are the names of the sailors that reserved the boat 100?

```
SELECT DISTINCT s.sailor_name  
FROM sailors AS s, reserves AS r  
WHERE s.sailor_id = r.sailor_id AND r.boat_id=100;
```

What are the names of the sailors that reserved at least one boat?

```
SELECT DISTINCT s.sailor_name  
FROM sailors AS s, reserves AS r  
WHERE s.sailor_id = r.sailor_id;
```

Using three tables

What are the names of the sailors that reserved a red boat?

```
SELECT DISTINCT s.sailor_name  
FROM sailors AS s, boats AS b, reserves AS r  
WHERE s.sailor_id = r.sailor_id AND b.boat_id = r.boat_id AND b.color = 'red';
```

More examples...

What are the names of the sailors that reserved a green or a red boat?

Using a Join of three tables:

```
SELECT s.sailor_name
FROM sailors AS s, boats AS b, reserves AS r
WHERE s.sailor_id = r.sailor_id
AND b.boat_id = r.boat_id
AND (b.color = 'red' or b.color = 'green');
```

Using a Union:

```
SELECT s.sailor_name
FROM sailors AS s, boats AS b, reserves AS r
WHERE s.sailor_id = r.sailor_id AND b.boat_id = r.boat_id AND b.color = 'red'
UNION
SELECT s.sailor_name
FROM sailors AS s, boats AS b, reserves AS r
WHERE s.sailor_id = r.sailor_id AND b.boat_id = r.boat_id AND b.color = 'green';
```

Sub-queries

A select command may contain one or more select commands.

They can appear in the WHERE, HAVING, FROM clauses.

Example: What are the names of the sailors that reserved the boat 100?

```
SELECT s.sailor_name
FROM sailors AS s
WHERE s.sailor_id IN(
    SELECT r.sailor_id
    FROM reserves AS r
    WHERE r.boat_id = 100
);
```

Another example...

Example: What are the names of the sailors that didn't reserve a red boat?

```
SELECT s.sailor_name
FROM sailors AS s
WHERE s.sailor_id NOT IN(
    SELECT r.sailor_id
    FROM reserves AS r
    WHERE r.boat_id IN(
        SELECT b.boat_id
        FROM boats AS b
        WHERE b.color = 'red'
    )
);
```

Aggregation operators

- COUNT - The number of values in the column.
- SUM - The sum of all values in the column.
- AVG - The average of all values in the column.
- MAX - The maximum value in the column.
- MIN - The minimum value in the column.

Examples

What is the average age of all the sailors?

```
SELECT AVG(s.age)
FROM sailors AS s;
```

What is the age of the oldest sailor?

```
SELECT MAX(s.age)
FROM sailors AS s;
```

What is the number of sailors?

```
SELECT COUNT(*)
FROM sailors;
```

Grouping and Filtering

```
SELECT [DISTINCT] select-list  
FROM from-list  
[WHERE qualification]  
[GROUP BY grouping-list]  
[HAVING group-qualification];
```

GROUP BY - Operator that gathers lines with equal values in a unique group, in the columns of the grouping-list.

HAVING - Filters the groups that don't satisfy the group-qualification.

Grouping and Filtering Examples

What is the age of the youngest sailor for each rating?

```
SELECT s.rating, MIN(s.age)
FROM sailors AS s
GROUP BY s.rating;
```

What are the ages of the youngest sailors for each rating with more than one sailor?

```
SELECT s.rating, MIN(s.age)
FROM sailors AS s
GROUP BY s.rating
HAVING COUNT(*) > 1;
```

Ordering

```
SELECT [DISTINCT] select-list  
FROM from-list  
[ORDER BY grouping-list];
```

The ORDER BY sorts the tuples in ascending order by default. To sort them descending we just have to add the DESC keyword.

Example: List by alphabetical order the names of sailors that reserved the boat 100.

```
SELECT s.sailor_name  
FROM sailors AS s, reserves as r  
WHERE s.sailor_id = r.sailor_id AND r.boat_id = 100  
ORDER BY s.sailor_name;
```

```
SELECT s.sailor_name  
FROM sailors AS s, reserves as r  
WHERE s.sailor_id = r.sailor_id AND r.boat_id = 100  
ORDER BY s.sailor_name DESC;
```

Database Design

How should we organize the data in a database?

Which tables should we create? Which columns should each table have?

Consider a relation (table) containing the students of a university:

full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

Is this a good design?

Redundancy

full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

Our database design contains a lot of redundant data!

Design Anomalies

Redundant data introduces multiple design anomalies:

full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

Can you identify what the specific problems are?

- Insertion anomaly: We can only change the name of a subject, like changing Math to Maths, after a student has enrolled in the class
- Deletion anomaly: When we delete a student coming from a city with no other students, we lose all information regarding that city.
- Update anomaly: When we change the student address, we have to change it for all rows regarding the same student

Database Normalization

A set of rules, guidelines and techniques that are used while designing a database

- Remove all the anomalies
- Distribute the data amongst different related tables
- Efficient and effective queries
- Removes all the duplication and incorrect data issues

Normalization helps us with having a well-designed database!

Functional Dependencies

Understanding the functional dependencies of our data is an important part of database design and the normalization process

A FD is a constraint between two sets of attributes in a relation:

full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

We can say that birth_date is functionally dependent on full_name as the birthdate of a student is determined by his name. By knowing the student name, we can determine its birthdate.

Functional Dependencies

The nature of our data in the real world is what determines the functional dependencies

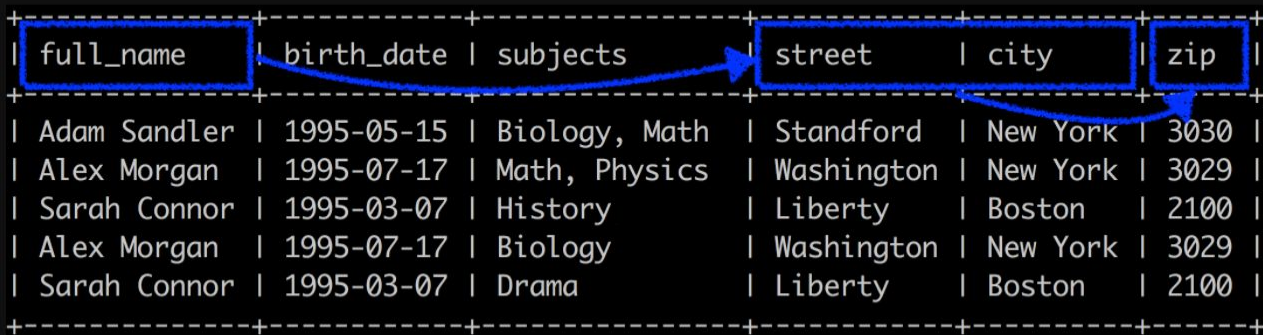
full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

Let's decide that for our university only one address will be used for each student, even if they actually have two homes. Then we can say that street, city and zip are functionally dependent on full_name

Can you identify any other functional dependencies in this relation?

Transitive Dependencies

Assuming the zip code is unique for a given city and street name:



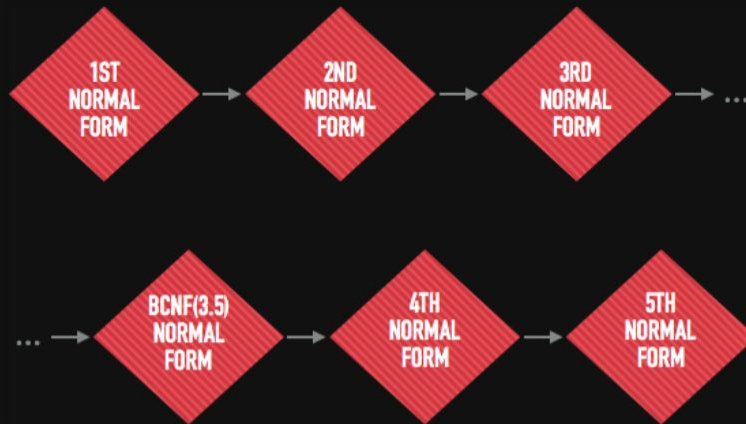
full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

We can say that zip is functionally dependent on street, city, which in turn are functionally dependent on full name

The attribute zip is said to be transitively dependent on full name!

Normal Forms

Normalization is divided into the following normal forms:



Usually we do not apply all the normal forms to get the database design done. The most used form is 3NF, 4NF and 5NF are ignored in most of the cases, but it depends on the design specific needs.

First Normal Form

full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

- Each row should be uniquely identified by a primary key ✓
- There should not be any repeating columns and each column should contain single values only ✗

First Normal Form

The first rule is satisfied by the <full_name, subjects> composite key.

But the subjects property breaks the second rule!



full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology, Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math, Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

First Normal Form

What if we replace the subjects column with two columns, named subject1 and subject2?

full_name	birth_date	subject1	subject2	street	city	zip
Adam Sandler	1995-05-15	Biology	Math	Standford	New York	3030
Alex Morgan	1995-07-17	Math	Physics	Washington	New York	3029
Sarah Connor	1995-03-07	History	Drama	Liberty	Boston	2100
Alex Morgan	1995-07-17	Biology	NULL	Washington	New York	3029

- There should not be any repeating columns and each column should contain single values only 🚫

First Normal Form

To satisfy 1NF, we must have only one subject column and for each student create as many rows as the subjects he is enrolled in

full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology	Standford	New York	3030
Adam Sandler	1995-05-15	Math	Standford	New York	3030
Alex Morgan	1995-07-17	Physics	Washington	New York	3029
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Alex Morgan	1995-07-17	Math	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

Data redundancy can increase, but each row will be unique

1NF does not provide a good database design, it is just an intermediate step!

Second Normal Form



full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology	Standford	New York	3030
Adam Sandler	1995-05-15	Math	Standford	New York	3030
Alex Morgan	1995-07-17	Physics	Washington	New York	3029
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Alex Morgan	1995-07-17	Math	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

- Data has to be 1NF ✓
- No partial dependencies on the primary key, all non-key attributes have to be dependent on all key attributes ✗

Second Normal Form



full_name	birth_date	subjects	street	city	zip
Adam Sandler	1995-05-15	Biology	Standford	New York	3030
Adam Sandler	1995-05-15	Math	Standford	New York	3030
Alex Morgan	1995-07-17	Physics	Washington	New York	3029
Alex Morgan	1995-07-17	Biology	Washington	New York	3029
Alex Morgan	1995-07-17	Math	Washington	New York	3029
Sarah Connor	1995-03-07	History	Liberty	Boston	2100
Sarah Connor	1995-03-07	Drama	Liberty	Boston	2100

- No partial dependencies on the primary key, all non-key attributes have to be functional dependent on all key attributes ✖
- birthdate, street, city and zip are non-key attributes that do not depend on the key-attribute subject

Second Normal Form

To satisfy 2NF, we must remove the subject attribute from the relation..

full_name	birth_date	street	city	zip
Adam Sandler	1995-05-15	Standford	New York	3030
Alex Morgan	1995-07-17	Washington	New York	3029
Sarah Connor	1995-03-07	Liberty	Boston	2100

..and create a new subjects relation:

full_name	subjects
Adam Sandler	Biology
Alex Morgan	Physics
Sarah Connor	History
Alex Morgan	Biology
Sarah Connor	Drama
Adam Sandler	Math
Alex Morgan	Math

2NF still does not provide a good database design, it is just an intermediate step!

Third Normal Form

full_name	birth_date	street	city	zip
Adam Sandler	1995-05-15	Standford	New York	3030
Alex Morgan	1995-07-17	Washington	New York	3029
Sarah Connor	1995-03-07	Liberty	Boston	2100

full_name	subjects
Adam Sandler	Biology
Alex Morgan	Physics
Sarah Connor	History
Alex Morgan	Biology
Sarah Connor	Drama
Adam Sandler	Math
Alex Morgan	Math

- Data has to be 2NF 
- No transitive dependencies allowed, every attribute has to be dependent only on key attributes 

Third Normal Form

full_name	birth_date	street	city	zip
Adam Sandler	1995-05-15	Stanford	New York	3030
Alex Morgan	1995-07-17	Washington	New York	3029
Sarah Connor	1995-03-07	Liberty	Boston	2100

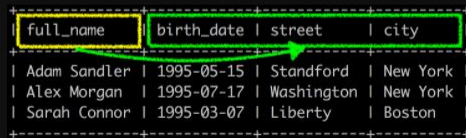
full_name	subjects
Adam Sandler	Biology
Alex Morgan	Physics
Sarah Connor	History
Alex Morgan	Biology
Sarah Connor	Drama
Adam Sandler	Math
Alex Morgan	Math

- No transitive dependencies allowed, every attribute has to be dependent only on key attributes 🚫

zip depends on non-key attributes city, street, which in turn depend on the key attribute full_name

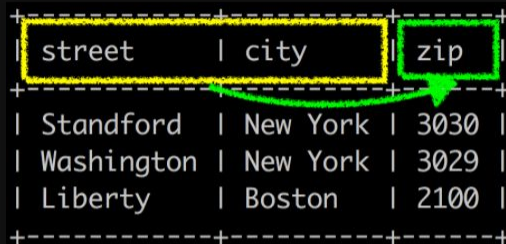
Third Normal Form

To satisfy 3NF, we must remove the zip attribute from the relation..



full_name	birth_date	street	city
Adam Sandler	1995-05-15	Standford	New York
Alex Morgan	1995-07-17	Washington	New York
Sarah Connor	1995-03-07	Liberty	Boston

..and create a new zip relation:



street	city	zip
Standford	New York	3030
Washington	New York	3029
Liberty	Boston	2100

we can not select zip code as a primary key on the zip table if two different streets can have the same zip

3NF is a satisfactory database design for this set of data!

How far should I normalize?

There is no definitive answer and it really depends on the data

3NF clearly didn't get rid of all the redundant data:

full_name	birth_date	street	city
Adam Sandler	1995-05-15	Standford	New York
Alex Morgan	1995-07-17	Washington	New York
Sarah Connor	1995-03-07	Liberty	Boston

street	city	zip
Standford	New York	3030
Washington	New York	3029
Liberty	Boston	2100

But how often will city and street names change or disappear?

{ Exercise }

Database modeling

The local library needs a software to manage their book loans.

Typical enquiries include:

"Do we have any books by Tolstoi?"

"How many books are out on loan?"

"Which company published *O Primo Basílio*?"

"Which users have borrowed books published before 1974?"

Stored Procedures

It is a program that is executed by a single SQL command.

It is executed in the RDBMS, which means:

- An application's logic is encapsulated;
- It is reusable by several applications;
- And some other advantages...

Some disadvantages:

- Harder to implement for complex logic;
- Harder to maintain;
- Application's logic becomes separated from the application.

Stored Procedure Syntax

```
USE boat_booking;
DELIMITER //
CREATE PROCEDURE get_reserves_by_boat_and_sailor
(IN boat VARCHAR(255), IN sailor VARCHAR(255))
BEGIN
SELECT DISTINCT * FROM reserves
WHERE reserves.bid = (
    SELECT id FROM boats WHERE boats.name = boat
) AND reserves.sid = (
    SELECT id from sailors WHERE sailors.name = sailor
);
END //
DELIMITER ;
```

```
// Call the procedure
CALL get_reserves_by_boat_and_sailor('Titanic', 'Nuno');
```

Triggers

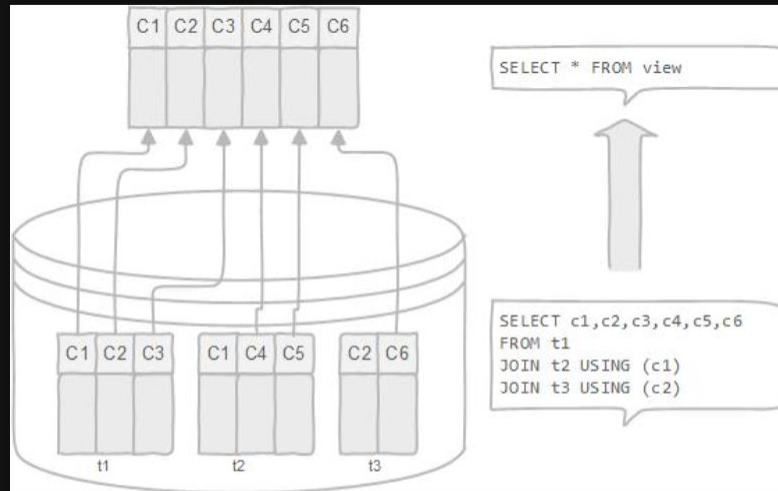
It is an automatically-invoked procedure in response to specific actions in the database.

Example:

```
DELIMITER //
CREATE TRIGGER before_rating_update
  BEFORE UPDATE ON sailors
  FOR EACH ROW
BEGIN
  INSERT INTO sailors_rating_history (sid, rating, day)
  VALUES (NEW.id, NEW.rating, CURDATE());
END //
DELIMITER ;
```

Views

A view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.



Create View Example

```
// Have a look at the mysql server users table
USE mysql;
SELECT * from user;

// Create a new view to simplify inspecting users
USE test;
CREATE VIEW my_users AS SELECT Host, User, authentication_string FROM mysql.user;

// Have a look at the mysql server users
SELECT * FROM my_users;
```

Concurrency

The concurrent execution of programs is essential for a good database performance. The same principles of parallel computing, that we discussed in the JAVA classes, apply!

Where will we have concurrency problems in databases?



Transactions

Transactions provide an all-or-nothing proposition, stating that each work-unit performed in a database must either complete in its entirety or have no effect whatsoever

The system must isolate each transaction from other transactions, results must conform to existing constraints in the database, and transactions that complete successfully must be written to durable storage

Example that should be wrapped in a database transaction: Making a money transfer.

- 1. read(A)
- 2. $A := A - 50$
- 3. write(A)
- 4. read(B)
- 5. $B := B + 50$
- 6. write(B)

Pattern used by Transactions

- Begin the transaction
- Execute a set of data manipulations and/or queries
- If no errors occur then commit the transaction and end it
- If errors occur then rollback the transaction and end it

Auto Commit

MySQL automatically commits all statements that are not part of an explicit transaction, making the results of any UPDATE or INSERT statement immediately stored to disk to make them permanent

```
// Inspect the state of the autocommit session(@@) variable
SELECT @@autocommit;

// Set the session variable autocommit to false
SET autocommit=0;
SELECT @@autocommit;
```

Successful Transaction

```
// Create a new table using InnoDB engine, which supports transactions
USE test;
CREATE TABLE test(num INTEGER NOT NULL) engine=InnoDB;

// Turn off auto commit
SET autocommit=0;

// Execute transaction, perform queries from other mysql connections
START TRANSACTION;
INSERT INTO test VALUES (1, 2, 3);
INSERT INTO test VALUES (4, 5);
COMMIT;
```

Transaction With Errors

```
// Truncate table test
TRUNCATE test;

// Execute transaction, perform queries from other mysql connections
START TRANSACTION;
INSERT INTO test VALUES (1, 2, 3);
INSERT INTO test VALUES (4, 5);

// Some error occurs.... let's rollback

ROLLBACK;
```

ACID Properties of Transactions

Transactions have the following properties:

- Atomicity
- Consistency
- Isolation
- Durability

Atomicity

State changes are atomic. Either they execute in full or they don't execute at all.

Why transactions fail:

- RDBMS finishes it due to an anomaly during execution;
- System crash (ex: power failure);
- Unexpected situation occurs and aborts (ex: cannot access the disc).
- Durability

Function: Changes made in an active transaction are kept in a transaction log and, in case of abort, all changes are undone.

Consistency

Ensures that any transaction will bring the database from a valid state to another valid state.

Function: Any data that is written to the database must be validated according to all restrictions (keys, foreign keys, cascades, triggers, etc.).

NOTE: It does not guarantee the correct execution of a transaction in the ways that the programmer intended (at the application level).

Isolation

It guarantees that concurrent execution of transactions results in a system state that would be obtained if transactions were executed serially

Function: Makes transactions only see changes performed by other successfully finished transactions.

Durability

Once a transaction is successfully finished, all changes are immutable, surviving any kind of fault.

Function: Data about changes of each committed transaction is kept in a log. In case of any fault, any changes that were not saved to the disc are remade.

Database Indexes

How to make data access more efficient?

If we have a database with many tables, each with thousands of rows, how can we make data access faster?

If we know beforehand what will be the most used queries, we can prepare the RDBMS to better perform such queries



Example: "Who are the sailors that reserved the boat with ID = X?"

Using Indexes

Indexes allow the RDBMS to speed up data access, allowing it to efficiently find all rows that match a given column in a query. By doing so, the RDBMS only has to walk through a subset of the table to find the exact matches, instead of walking through the entire table and checking every single row.

In general, indexes are smaller than the table which they index.

It makes reading (select queries and where clauses) faster, but the writing (update and insert statements) becomes slower.

Creating Indexes

```
//creating index  
CREATE INDEX index_name ON table_name(column_name);  
  
//dropping index  
DROP index_name ON table_name;
```

Primary keys are already indexed by default

When to avoid indexes?

- When they concern a small table (few tuples);
- When they concern tables that have frequent insert and/or update operations;
- When they concern columns of tables with many NULL values.